

Cross-Residual Graph Neural Networks for Biomolecular Graph Representation Learning

Jingchao Wang¹, Wei Huang¹, Xuelin Hu^{2,*}, Xiaoqin Fu²

¹*School of Computer Science, Zhongyuan University of Technology
Zhengzhou 450000, China*

²*School of Railway Communication and Signaling Engineering
Liuzhou Railway Vocational Technical College, Liuzhou, Guangxi, China*

static@zut.edu.cn, huangwei2913@gmail.com, huxuelinai@gmail.com
huxl@ltzy.edu.cn, xiaoqin.fu@qq.com, fuxiaoqin@ltzy.edu.cn

*Corresponding author: Xuelin Hu

ORCID: Xuelin Hu (0000-0002-4475-3034); Xiaoqin Fu (0009-0003-5123-8393)

Abstract

Graph neural networks are increasingly used in biomolecular and plant-related graph learning, yet deep message passing can blur local structural cues while simple skip connections do not test richer information reuse. This limitation is especially relevant for biological graph classification, where predictive signals often depend on both local motifs and graph-level context. We therefore study Cross-Residual Graph Neural Networks (CR-GNN), a compact model family that compares five information-flow patterns under a shared backbone: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN. Experiments on a protein-oriented benchmark centered on PROTEINS, DD, and ENZYMES, with MUTAG, AIDS, and Mutagenicity as supplementary robustness checks, show a consistent picture. Residual reuse is the strongest default strategy, while cross-residual routing usually improves over plain stacking and delivers dataset-dependent gains on selected graphs. These results position CR-GNN as a practical biomolecular graph representation framework and clarify when cross-path interaction is useful in biologically motivated graph learning.

1 Introduction

Graph neural networks (GNNs) are now a core tool for structure-aware learning on molecules, proteins, and biological interaction systems [1–3]. In the biological domain, graph representations are valuable because they preserve both local neighborhood structure and larger relational context, which is critical for tasks such as protein function prediction, protein–protein interface analysis, enzyme classification, and biomolecular interaction modeling. Recent studies have already shown concrete biological impact: graph convolutional models have been used for structure-based protein function prediction [4], large-scale multispecies protein annotation [5], and protein–protein interface learning [6]. The same trend is increasingly visible in plant com-

putational biology, where GNN-style modeling is being explored for plant protein identification, functional annotation, and biological network analysis [7–9].

Despite this progress, deep biological graph learning remains difficult. Repeated message passing can over-smooth node states, weaken discriminative local motifs, and reduce intermediate diversity, especially when the graph label depends on multiple coordinated substructures rather than on one dominant pattern. This issue is especially relevant in protein-related graphs and plant-oriented graph inference, where useful evidence may be distributed across local regions, cross-scale interactions, and graph-level summaries. A model that only stacks layers sequentially may underuse such complementary information, while a simple residual skip only re-injects the same branch history and does not test whether cross-branch exchange or graph-level feedback can preserve more useful context.

Modern GNN research has proposed several mechanisms to improve deep training, including residual connections [10, 11], normalization [12, 13], edge dropout [14], and multi-scale readout designs such as Jumping Knowledge [15]. These methods improve optimization or preserve multi-depth features, but most still operate inside a single branch. In parallel, graph-level pooling and hierarchical modules improve global summarization, yet they do not directly test whether exchanging intermediate states across branches or reusing graph summaries across block boundaries yields a better trade-off for biomolecular graph inference.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a unified family of graph classification models that isolates three kinds of information reuse: intra-branch residual accumulation, cross-branch node-state exchange, and graph-level summary propagation across sequential blocks. Rather than presenting one oversized architecture, we formulate a controlled model family that includes plain, residual, and cross-residual variants under the same message-passing backbone. This makes it possible to ask a more useful question than whether one model wins everywhere: when does cross-residual interaction help beyond a simpler residual baseline on biomolecular graphs, and when does residual reuse remain the better default for protein-oriented graph inference?

This framing also clarifies the scope of the study. The present work does not claim to solve full integrative omics modeling or novel biological discovery directly. Instead, it focuses on graph representation learning for a protein-oriented benchmark package spanning PROTEINS, DD, and ENZYMES, and studies how information reuse can preserve biologically meaningful structure more effectively than plain stacking. In this sense, the paper should be read as a biomolecular graph learning study with a technically motivated extension path toward plant-related graph inference settings in which graph topology can later be combined with richer functional or omics annotations.

The contributions of this work are summarized as follows:

- We formalize CR-GNN as a compact architecture family that compares five biologically relevant graph classification models under one shared backbone: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN.
- We design a biologically focused evaluation protocol built around a protein-oriented benchmark core (PROTEINS, DD, ENZYMES) plus supplementary robustness datasets, and

we use a controlled stratified 5-fold setting to make the architectural comparison more rigorous.

- We show that cross-residual design is useful but not universally dominant: residual reuse remains the strongest default, while cross-residual routing consistently improves over plain stacking and yields selective gains that clarify when node-level or graph-level information reuse is most helpful.

2 Related Work

The motivation of CR-GNN is not only architectural but also domain-specific: biological graph learning often requires preserving local structural cues together with broader graph context. We therefore review the literature from six connected perspectives, starting with biomolecular and plant applications before moving to graph classification, message passing operators, deep-training challenges, hybrid architectures, and cross-layer aggregation.

2.1 GNNs in Biomolecular and Plant Graph Learning

Biological graph learning has moved well beyond generic graph benchmarks. In biomolecular settings, GNNs have already been used for structure-based protein function prediction [4], large-scale protein function prediction across species [5], and protein-protein interface pattern discovery [6]. These studies show that graph models are practically relevant when the target depends on coordinated structural signals rather than on isolated features. The same pattern is now emerging in plant computational biology. Recent work applies graph-based modeling to plant vacuole protein identification [7], plant biological-network analysis [8], and plant functional annotation from knowledge graphs [9]. This literature does not validate our exact cross-residual design, but it makes the application setting concrete: in both biomolecular and plant tasks, the model often needs to preserve multi-scale biological information rather than only deepen a single message-passing chain.

Benchmark choice matters in this context. TU datasets such as PROTEINS, DD, and ENZYMES remain widely used because they are easy to reproduce and still expose meaningful differences in graph scale, sparsity, and classification difficulty [16]. For the present work, these datasets provide a stable protein-oriented evaluation core while keeping the narrative aligned with real biological graph-learning use cases.

2.2 Graph Neural Networks for Graph Classification

Graph-level prediction requires aggregating node-level features into fixed-size graph representations. Early graph neural networks [17] laid the foundation for learning on graph-structured data through recursive message passing. The Message Passing Neural Networks (MPNN) framework [2] unified various GNN architectures under a common formalism, identifying message and vertex update functions as core components.

For graph classification, readout functions play a crucial role in converting node embeddings to graph-level representations. Simple approaches apply global pooling operations such as mean

or sum pooling [18]. However, these methods may not adequately capture hierarchical structural information. To address this, differentiable pooling methods have been proposed: DiffPool [19] learns node cluster assignments to coarsen graphs hierarchically, SAGPool [20] employs self-attention to identify salient nodes, and LocalPool [21] enables sparse hierarchical classification. Recent surveys provide comprehensive overviews of graph pooling techniques [22, 23]. While these approaches achieve strong performance, they primarily focus on pooling design rather than on how residual information should be reused across node-level and graph-level computation stages.

2.3 Message Passing Architectures and Operators

Various message passing operators have been developed, each imposing different inductive biases on neighborhood aggregation. Graph Convolutional Networks (GCN) [1] approximate spectral convolutions through normalized adjacency matrices, performing low-pass filtering that smooths node features. GraphSAGE [3] samples and aggregates features from local neighborhoods using mean, LSTM, or pooling aggregators, enabling inductive learning on unseen graphs. Graph Isomorphism Networks (GIN) [24] use injective aggregation functions to achieve maximum discriminative power, matching the expressive power of the Weisfeiler-Lehman test.

Attention mechanisms have been widely incorporated to weight neighbor contributions adaptively. Graph Attention Networks (GAT) [25] compute attention coefficients for each neighbor, allowing nodes to focus on relevant information. Personalized Graph Neural Networks (PGNN) [26] further extend attention mechanisms for personalized aggregation. Recent work generalizes Transformer architectures to graphs [27] and incorporates edge directionality [28, 29] for improved modeling of directed and heterophilic graphs.

Other operator variants include ARMA filters [30] for flexible frequency response, MixHop [31] for mixing multi-hop neighborhoods, and anti-symmetric convolutions [32] for stable deep architectures. While these operators are typically studied and deployed individually within homogeneous architectures, a critical gap remains: the literature provides much less comparison of how different residual information-flow mechanisms behave across operator families under a unified training protocol.

2.4 Deep GNN Training and Representation Degradation

Training deep GNNs faces fundamental challenges due to representation degradation. The *over-smoothing* phenomenon causes node representations to converge toward indistinguishable values as depth increases [33–35]. Theoretical analysis [35] provides non-asymptotic bounds on over-smoothing, revealing fundamental limitations in expressive power for deep architectures. Additionally, *over-squashing* [36, 37] occurs when long-range dependencies cannot be effectively propagated due to graph bottlenecks, further constraining model performance.

To enable deep training, various techniques have been proposed. Residual connections [10], originally developed for computer vision, have been adapted to GNNs through residual gated graph convolutions [11]. Normalization methods such as PairNorm [12] and GraphNorm [13] alleviate over-smoothing by normalizing node features, while learnable normalization [38] adapts

to different graph distributions. Regularization techniques like DropEdge [14] randomly drop edges during training to prevent overfitting and improve generalization.

Advanced architectures attempt to train extremely deep networks: techniques for training 1000-layer GNNs [39] and simplified deep GCN approaches [40] demonstrate that depth can improve performance with proper design. However, these methods primarily focus on single-path deepening and do not explicitly compare node-level residual reuse with graph-level residual propagation as two distinct ways of preserving information at depth.

2.5 Multi-Operator and Hybrid Architectures

Combining multiple aggregation schemes or architectural components has shown promise in enhancing representational capacity. Multi-hop attention [41] enables information flow across longer distances, while ARMA filters [30] provide flexible frequency responses beyond polynomial filters. Hybrid architectures incorporate edge directionality [28, 29] to capture directed relationships, and anti-symmetric convolutions [32] improve stability in deep networks.

Surveys on attention-based GNNs [42] and graph embeddings [43] provide comprehensive taxonomies of architectural variants. DenseGNN [44] connects all layers densely, similar to DenseNet in computer vision, to maintain information flow. Hierarchical pooling methods [19, 45] create multi-scale representations through graph coarsening. However, most existing hybrid approaches emphasize component combination rather than a systematic comparison of where residual information should be injected and reused. Our work addresses this gap by contrasting node-level residual reuse, graph-level residual propagation, and cross-branch information exchange within a unified architecture family.

2.6 Cross-Layer and Multi-Scale Aggregation

Cross-layer connections aim to capture multi-scale information by aggregating features from different depths. Jumping Knowledge (JK) networks [15] adaptively select which layer’s representation to use for each node, enabling flexible neighborhood range selection. Graph U-Net [46] adopts encoder-decoder architecture with skip connections between corresponding layers.

Propagation-based methods improve information flow across multiple hops: APPNP [47] combines personalized PageRank with neural predictions, while PairNorm [12] and DropEdge [14] mitigate over-smoothing through normalization and regularization. Recent work on over-smoothing [34, 48] provides comprehensive analysis and solutions.

Despite these advances, most cross-layer methods operate within a single homogeneous branch using one type of operator. A fundamental limitation is that they rarely distinguish between residual reuse at the node-representation level and residual reuse at the graph-summary level. Our framework focuses on this distinction by introducing node-level and graph-level cross-residual mechanisms within a common graph classification design space. This perspective enables a more direct comparison of how different information-flow patterns affect robustness, depth tolerance, and graph-level discrimination in biological graph classification. With that context in place, we next describe the benchmark package used to test the architecture under a controlled biological setting.

3 Task Definition

We formally define the graph classification problem and establish notation for the five-model comparison used throughout the paper. Rather than changing many components at once, we keep the graph classification setting fixed and study how different residual information-flow mechanisms affect graph-level prediction under a common formulation.

3.1 Graph Representation and Notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, where d_{in} is the input feature dimension. The node features for the entire graph are collected in a matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$, where the i -th row corresponds to $\mathbf{x}_i$.

The graph structure is represented using an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ if there exists an edge $(v_i, v_j) \in \mathcal{E}$, and $A_{ij} = 0$ otherwise. For undirected graphs, the adjacency matrix is symmetric ($\mathbf{A} = \mathbf{A}^\top$). Optionally, edge features can be represented as $\mathbf{e}_{uv} \in \mathbb{R}^{d_e}$ for each edge $(u, v) \in \mathcal{E}$, or collected in a tensor $\mathbf{E} \in \mathbb{R}^{n \times n \times d_e}$.

3.2 Graph Classification Problem

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$ consisting of N graphs, where each graph $\mathcal{G}_i$ is associated with a label $y_i \in \mathcal{Y}$, our objective is to learn a mapping function $f_\theta : \mathcal{G} \rightarrow \hat{y}$ that predicts the label $\hat{y} \in \mathcal{Y}$ for an unseen graph. The label space $\mathcal{Y}$ depends on the task type:

- **Binary classification:** $\mathcal{Y} = \{0, 1\}$ (e.g., molecular toxicity prediction)
- **Multi-class classification:** $\mathcal{Y} = \{1, 2, \dots, C\}$ (e.g., graph categorization into C classes)
- **Multi-label classification:** $\mathcal{Y} \subseteq \{1, 2, \dots, C\}$ (e.g., a graph may belong to multiple categories simultaneously)

The function f_θ is parameterized by neural network weights θ and operates by first learning node-level representations through message passing, then aggregating these into a graph-level representation, and finally mapping this representation to the output label space.

3.3 Main Comparison Setting

The main paper compares five architectures under the same graph-classification pipeline:

- **PlainGNN:** sequential message passing without explicit state reuse
- **NodeResGNN:** single-branch node-level residual reuse
- **NodeCrossGNN:** two-branch node-level cross exchange
- **GraphResGNN:** graph-level residual propagation across sequential blocks
- **GraphCrossGNN:** graph-level cross-summary exchange across paired blocks

The comparison is designed to isolate *where* information is reused: inside one branch, across parallel branches, or through pooled graph summaries passed between blocks. This design makes the main experimental question precise: when does cross-path interaction help beyond plain stacking and beyond a simpler residual baseline?

3.4 Message Passing with Multiple Operators

Let Φ denote a message-passing operator. For a chosen backbone, node representations are updated over L layers. At layer ℓ , the representation $\mathbf{h}_i^{(\ell)}$ of node v_i is computed as:

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_{\Phi} \left(\left\{ \mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell)} \right) + \mathbf{b}^{(\ell)} \right) \quad (1)$$

where AGGREGATE_{Φ} is the aggregation function induced by Φ , $\mathbf{W}^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ are learnable parameters, and σ is a non-linear activation function. In the experiments reported later, all five models are compared under the same GCN-style backbone so that the observed differences are attributable to information-flow design rather than to operator changes.

3.5 Graph-Level Representation Learning

After L layers of message passing, a readout function R aggregates node-level representations into a graph-level embedding $\mathbf{h}_{\mathcal{G}} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{h}_{\mathcal{G}} = R \left(\left\{ \mathbf{h}_i^{(L)} : i = 1, \dots, n \right\} \right) \quad (2)$$

Common readout functions include mean, max, and sum pooling. In our framework, graph-level representations may also be reused across successive computation blocks, enabling later stages to incorporate coarse whole-graph context in addition to local message-passing features.

3.6 Learning Objective

The graph-level representation is passed through a classifier (typically a linear layer followed by softmax) to produce the predicted label distribution:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}} \mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}) \quad (3)$$

where $\hat{\mathbf{y}} \in \mathbb{R}^C$ (for multi-class classification) is the predicted probability distribution over C classes, and $\mathbf{W}_{\text{clf}} \in \mathbb{R}^{C \times d_{\text{out}}}$, $\mathbf{b}_{\text{clf}} \in \mathbb{R}^C$ are classifier parameters.

The model is trained end-to-end by minimizing the cross-entropy loss function:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (4)$$

where $y_{i,c} \in \{0, 1\}$ is the ground-truth label indicator (one-hot encoding) for graph $\mathcal{G}_i$ and class c , and $\hat{y}_{i,c}$ is the predicted probability for class c . For binary classification, this reduces to the binary cross-entropy loss. With this common formulation fixed, the next section instantiates the five CR-GNN variants and specifies how each one reuses intermediate node states or graph summaries.

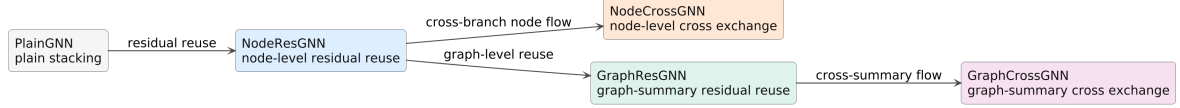


Figure 1: Main model comparison in this paper. The five architectures differ only in how intermediate node states or graph summaries are reused, which allows a controlled study of residual and cross-residual information flow.

4 Proposed Model

Based on the task formulation above, CR-GNN is defined as a compact family of graph classification architectures. All variants share the same input projection, stacked message-passing layers, graph-level pooling, and linear classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches.

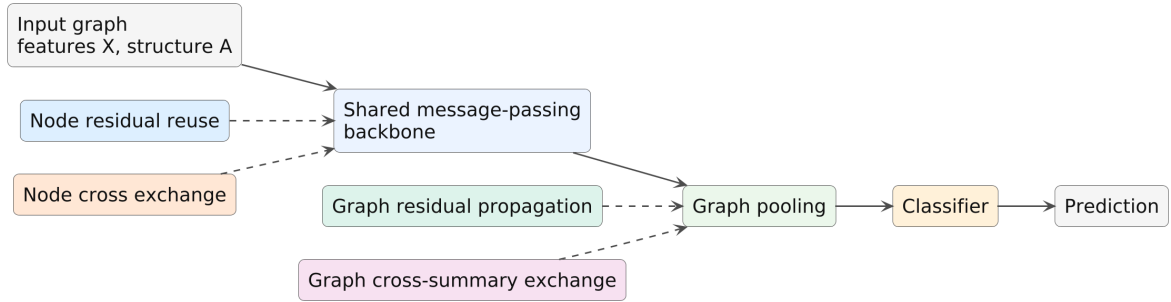


Figure 2: Schematic view of the CR-GNN pipeline. All models share the same graph-classification backbone, while node-level reuse and graph-level summary reuse define the residual and cross-residual variants.

4.1 Operator Factory and Shared Backbone

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. The model can be instantiated with one of several standard message-passing operators,

$$\Phi \in \{\text{GCN}, \text{GAT}, \text{Graph-Transformer}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d , every branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

followed by L hidden layers and a readout

$$\mathbf{g} = \text{Pool}(\mathbf{H}^{(L)}). \quad (6)$$

The final graph prediction is produced by

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{clf}} \psi(\mathbf{g}) + \mathbf{b}_{\text{clf}}, \quad (7)$$

where $\psi(\cdot)$ denotes regularized graph-level feature transformation before classification.

The main paper focuses on five architectures: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN. They differ only in how they update $\mathbf{H}^{(\ell)}$ and how they reuse graph-level embeddings across blocks.

4.2 Single-Branch and Node-Level Residual Blocks

PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell\left(\mathbf{H}^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (8)$$

This model serves as the reference point for judging whether residual information flow is beneficial.

NodeResGNN. The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell\left(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\right)\right)\right). \quad (9)$$

This mechanism is lightweight and preserves historical features without creating extra branches.

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell^{(1)}\left(\mathbf{H}_1^{(\ell)} + \tilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (10)$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell^{(2)}\left(\mathbf{H}_2^{(\ell)} + \tilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (11)$$

where $\tilde{\mathbf{H}}_k^{(\ell)}$ is the cached hidden state from the opposite branch before its current update. The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (12)$$

Compared with the single-branch residual block, NodeCrossGNN explicitly exchanges information between parallel pathways at every depth.

4.3 Graph-Level Residual Propagation

We further consider two higher-level architectures that compose several block models sequentially and pass graph embeddings across them.

GraphResGNN. This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\right), \quad t \in \{1, 2\}. \quad (13)$$

Inside each block, the graph embedding is broadcast back to node states before the next graph convolution. The design can be viewed as repeated graph-level residual refinement.

GraphCrossGNN. The strongest graph-level interaction appears in the cross-graph variant. Four blocks are organized as two sequential pairs. Within each stage, the two block outputs

are swapped and reused as the next stage’s graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}), B_{2t}(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}) \right), \quad (14)$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = (\mathbf{g}_2^{(t)}, \mathbf{g}_1^{(t)}). \quad (15)$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \quad (16)$$

In this variant, interaction occurs through exchanged pooled graph summaries rather than through direct node-tensor exchange.

4.4 Unified Training Objective

All model variants use the same graph classification objective:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c}, \quad (17)$$

with the same loss definition used for every architecture. The experiments in Section 6 therefore isolate architectural differences rather than changes in objective or evaluation protocol.

4.5 Algorithmic View

Table 1 summarizes the shared forward logic in a compact three-line format.

4.6 Design Implications

This architecture family suggests three practical hypotheses that are later examined experimentally. First, residual reuse should matter more consistently than simply adding more branches, because every successful variant still relies on repeated feature reuse. Second, node-level cross residual and graph-level residual passing are not interchangeable: the former exchanges intermediate node representations, whereas the latter only re-injects pooled graph summaries. Third, the usefulness of cross-branch fusion depends on dataset scale and operator bias; it is therefore necessary to analyze accuracy, stability, and efficiency jointly rather than reporting only the single best score. Appendix A further gives a simplified linearized derivation showing why residual reuse can slow the collapse associated with over-smoothing.

5 Datasets

To test the proposed architecture in a biologically meaningful setting, we organize the benchmarks into a unified two-layer package: a main biological benchmark that supports the paper’s protein-oriented narrative, and a supplementary robustness package that broadens structural coverage without diluting the biological focus. This section presents the active datasets together with task type, source, split protocol, and graph statistics.

Table 1: Shared forward procedure of the CR-GNN family.

Step	Procedure
1	Input graph features $\mathbf{X}$ and structure $\mathbf{A}$, choose depth L , and select one architecture from PLAINGNN, NODERESGNN, NODECROSSGNN, GRAPHRESGNN, and GRAPHCROSSGNN.
2	Apply the shared input projection to obtain initial hidden states.
3	If the model is PLAINGNN, update one branch sequentially for L layers without explicit state reuse.
4	If the model is NODERESGNN, add the cached previous hidden state before each layer update.
5	If the model is NODECROSSGNN, maintain two branches and inject the opposite branch’s cached state at every depth.
6	If the model is GRAPHRESGNN, execute sequential blocks and reuse the previous graph embedding as graph-level context.
7	If the model is GRAPHCROSSGNN, compute paired graph embeddings and swap them before the next block stage.
8	Pool the final node representations to obtain $\mathbf{g}$, then apply the graph-level classifier to produce logits $\hat{\mathbf{y}}$.
9	Return the prediction $\hat{\mathbf{y}}$ and the final graph embedding $\mathbf{g}$.

5.1 Unified Benchmark Package

The main biological benchmark consists of:

- **PROTEINS**: graph-level protein classification from the TU Dortmund collection [16, 49]
- **DD**: large sparse protein structure graph classification from the same TU collection [16]
- **ENZYMES**: enzyme-oriented multi-class graph classification from the same benchmark family [16]

The supplementary robustness package consists of:

- **MUTAG**
- **AIDS**
- **Mutagenicity**

This organization is deliberate. PROTEINS, DD, and ENZYMES provide the clearest protein-oriented evaluation core for the present study. The supplementary datasets remain useful as structural robustness checks, but they do not define the main biological claim.

5.2 Main Biological Datasets

With this package structure fixed, we next describe the three main datasets in more detail, because they carry the core biological interpretation used later in the experiments section.

PROTEINS PROTEINS contains graph representations of proteins, where nodes denote secondary structure elements and edges encode neighborhood relations between them. The task is binary graph classification. In this benchmark, the dataset contains 1,113 graphs, 2 classes, and 3 input features per node. The average graph has 39.06 nodes and 72.82 edges, although the size variation is substantial.

DD DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent amino acids or structure-related units linked by proximity-derived edges. The task is binary classification. DD is the largest and sparsest benchmark in the main package, with 1,178 graphs, 2 classes, and 89 input features. On average, graphs contain 284.32 nodes and 715.66 edges, with individual graphs reaching 5,748 nodes.

ENZYMES ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input features. It is particularly relevant here because it preserves an enzyme-oriented biological interpretation while remaining directly comparable to the other graph classification benchmarks in the study. The average graph has 32.63 nodes and 62.14 edges, and the class distribution is perfectly balanced across the six categories.

5.3 Unified Dataset Statistics

Tables 2 and 3 summarize all datasets used in the revised manuscript. The key point is that every dataset is now positioned explicitly either as part of the main biological package or as supplementary robustness evidence.

5.4 Data Loading

All active datasets are used through a unified benchmark interface [16, 49]. We keep preprocessing intentionally light and do not introduce task-specific feature engineering, graph augmentation, or edge redesign in the final protocol. This keeps the comparison focused on architectural information reuse rather than on dataset-specific preprocessing choices.

5.5 Splitting Protocol

All reported results follow the same stratified two-stage evaluation design:

- **Outer evaluation:** stratified 5-fold cross-validation at the graph level
- **Inner validation:** a stratified validation subset sampled from the training fold with validation ratio 0.1

The stratified protocol reduces class-imbalance artifacts and provides a stable basis for validation-based model selection. Each outer fold keeps a distinct but reproducible validation partition.

5.6 Evaluation Metrics

The primary evaluation metric is graph classification accuracy on the held-out test fold. For fold k with test set $\mathcal{T}_k$, the fold accuracy is

$$\text{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \quad (18)$$

Table 2: Unified summary of the main biological benchmark package.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	TU	classification	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	TU	classification	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	TU	classification	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 3: Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	TU	classification	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	TU	classification	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	TU	classification	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

Across the five folds, we report the mean accuracy

$$\overline{\text{Acc}} = \frac{1}{5} \sum_{k=1}^5 \text{Acc}_k \quad (19)$$

and the corresponding standard deviation

$$\sigma_{\text{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^5 (\text{Acc}_k - \overline{\text{Acc}})^2}. \quad (20)$$

The paper therefore reports each result as mean accuracy $\pm$ standard deviation over the five stratified folds.

6 Experiments

This section answers two questions. First, does cross-residual information flow improve graph classification beyond plain stacking? Second, when compared against simpler residual reuse, where does the cross design help and where does it remain weaker?

6.1 Experimental Settings

We begin with the shared protocol, because the later biological interpretation is only meaningful if model, data, and optimization choices are controlled consistently.

6.1.1 Model Set

We compare five architectures under the same GCN-style message-passing backbone:

1. **PlainGNN**: plain stacked propagation without explicit hidden-state reuse
2. **NodeResGNN**: single-branch node-level residual reuse
3. **NodeCrossGNN**: two-branch node-level cross residual exchange
4. **GraphResGNN**: graph-level residual refinement driven by pooled graph summaries
5. **GraphCrossGNN**: graph-level cross residual exchange across consecutive block pairs

This comparison isolates the effect of information-flow design while holding the operator family fixed.

6.1.2 Dataset Scope

The revised topic-facing discussion centers on the unified biological package introduced in Section 5:

- **Main biological package:** PROTEINS, DD, ENZYMES
- **Supplementary biological or chemical datasets:** MUTAG, AIDS, Mutagenicity

This package separates the protein-oriented main narrative from supplementary robustness evidence and keeps the main text aligned with the biological scope of the paper.

6.1.3 Data Protocol

With the benchmark scope fixed, we next define the common evaluation procedure applied to every model.

All reported results use stratified 5-fold graph-level evaluation. Inside each training fold, we construct a stratified validation subset with validation ratio 0.1. Model selection is based on validation loss, and the best checkpoint is evaluated on the held-out test fold.

The stratified design matters because it reduces class-distribution artifacts and exposes architectural differences more reliably.

6.1.4 Optimization

Once the split protocol is fixed, the remaining source of variation is the training configuration itself. All models are trained under the same optimization framework with validation-based model selection and conservative regularization. Plain and residual baselines follow the same common training template, while the cross-residual models are allowed modest stability-oriented adjustment within that same regime. This keeps the comparison fair while avoiding optimization failure from dominating the architectural comparison.

6.1.5 Reproducibility

All reported runs follow the same reproducible evaluation policy. The important point for the paper is that every architecture is tested under the same data protocol, the same operator backbone, and the same model-selection criterion.

6.2 Full-Suite Results

With the protocol fixed, we first report the full benchmark picture before narrowing to the topic-facing biological interpretation.

Tables 4–6 summarize the revised benchmark package over the active biological and supplementary datasets.

Table 4: Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

Table 5: Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486	0.8455 $\pm$ 0.0257	0.7814 $\pm$ 0.0235
NodeResGNN	0.7343 $\pm$ 0.0734	0.8690 $\pm$ 0.0354	0.7897 $\pm$ 0.0139
NodeCrossGNN	0.7397 $\pm$ 0.0676	0.8735 $\pm$ 0.0412	0.8033 $\pm$ 0.0159
GraphResGNN	0.7290 $\pm$ 0.0585	0.9160 $\pm$ 0.0121	0.8022 $\pm$ 0.0220
GraphCrossGNN	0.7129 $\pm$ 0.0539	0.8710 $\pm$ 0.0412	0.8029 $\pm$ 0.0182

Table 6: Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

The winner pattern is immediately informative. NodeCrossGNN wins on MUTAG and Mutagenicity, GraphResGNN wins on PROTEINS, ENZYMES, and AIDS, and NodeResGNN wins on DD. PlainGNN does not win any completed dataset in the reported benchmark suite.

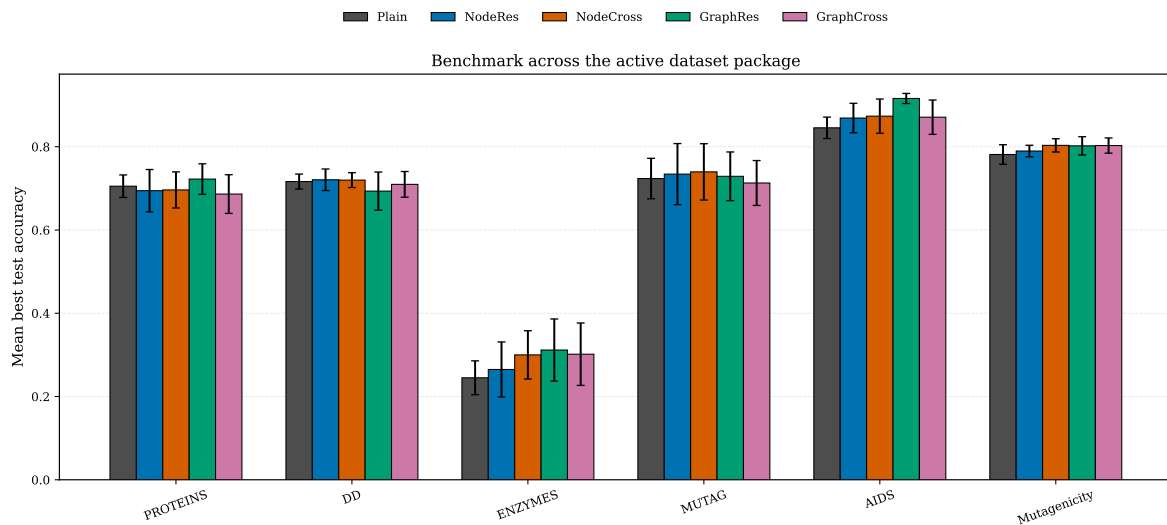


Figure 3: Benchmark over the active dataset package. Each bar reports mean best test accuracy, and error bars show the fold-level standard deviation. The current archive highlights the protein-oriented TU core together with the supplementary robustness pattern.

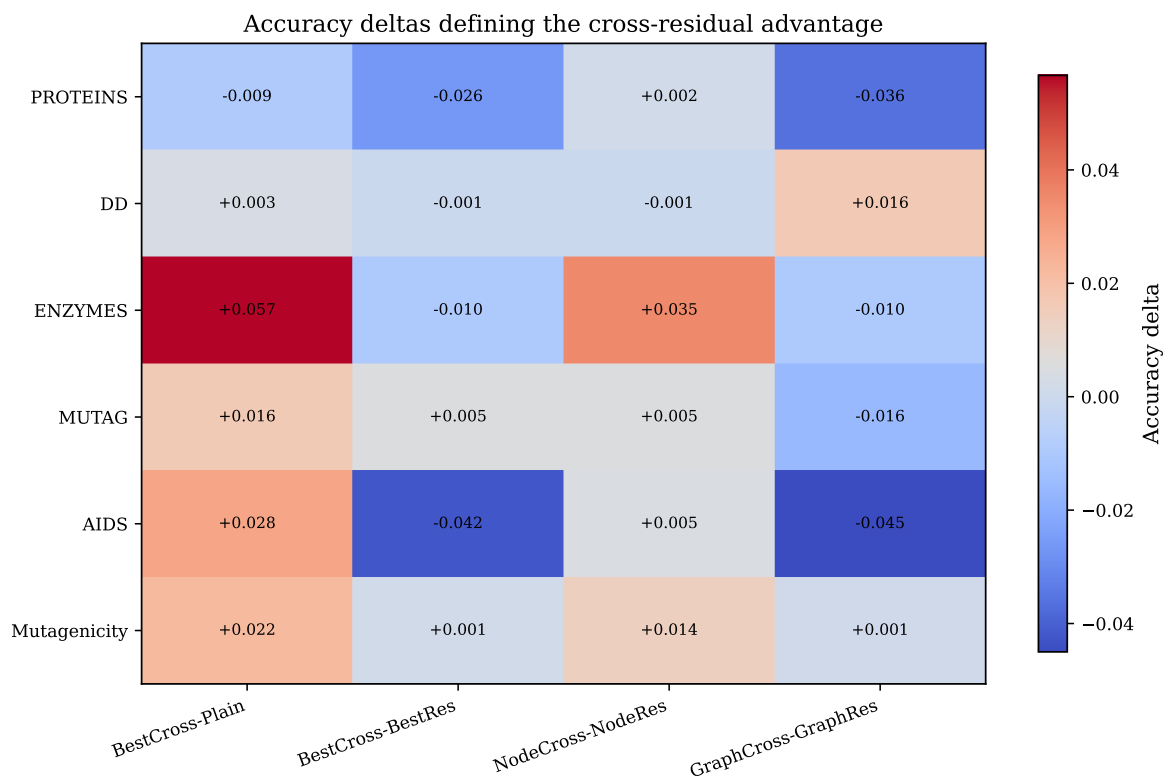


Figure 4: Accuracy deltas that define the practical value of cross-residual routing. Positive values indicate an advantage for the cross design, while negative values indicate that the corresponding plain or residual alternative remains stronger.

6.3 Main Findings

These aggregate results already suggest the central pattern, which can be stated more directly before examining the protein-oriented datasets one by one.

Residual reuse remains the strongest default family in the reported suite. Residual variants win PROTEINS, DD, ENZYMES, and AIDS. This makes residual reuse the safest default baseline under the final protocol. In particular, GraphResGNN is best on PROTEINS (0.7223 ± 0.0367), ENZYMES (0.3117 ± 0.0746), and AIDS (0.9160 ± 0.0121), while NodeResGNN is best on DD (0.7207 ± 0.0259).

Cross-residual design is selectively strong, not universally dominant. Cross variants still win completed datasets such as MUTAG and Mutagenicity. More importantly, the best cross variant beats PlainGNN on most completed datasets. Therefore, cross routing is clearly more than a design curiosity: it is a useful alternative information-flow mechanism. However, it still does not consistently beat the best residual baseline, so the evidence does not support the stronger claim that cross universally outperforms residual reuse.

Node-level cross is more reliable than graph-level cross. Across the completed datasets, NodeCrossGNN outperforms NodeResGNN more often than GraphCrossGNN outperforms GraphResGNN. This suggests that exchanging intermediate node-level states is the more robust cross-residual design in the present study, while graph-level cross routing is more dataset-sensitive.

Plain stacking is no longer a credible final baseline. PlainGNN remains competitive on PROTEINS and DD, but it never wins a dataset in the final suite and is usually surpassed by either a residual or a cross-residual variant. This is important for the paper narrative: the meaningful comparison is no longer cross versus plain, but cross versus the stronger residual family.

6.4 Topic-Facing Biological Interpretation

The full-suite pattern is still methodologically useful, but the topic-facing interpretation should be anchored in the protein-oriented core datasets. In that narrower sense, the strongest evidence comes from PROTEINS, DD, and ENZYMES, because they are the closest proxies in the current paper to protein- and enzyme-centered functional inference.

On **PROTEINS**, GraphResGNN is clearly strongest, indicating that repeated graph-level residual refinement is the most effective way to preserve useful global structure. The best cross model here is NodeCrossGNN at 0.6963 ± 0.0433 , which improves slightly over NodeResGNN but remains below both PlainGNN and GraphResGNN.

On **DD**, NodeResGNN and NodeCrossGNN are nearly tied, with NodeResGNN retaining a narrow lead. This indicates that node-level hidden-state reuse is important on large sparse protein graphs, but cross exchange does not produce a decisive improvement over simpler residual reuse.

On **ENZYMES**, cross variants become more attractive. GraphCrossGNN reaches 0.3017 ± 0.0750 and NodeCrossGNN reaches 0.3000 ± 0.0580 , both clearly above PlainGNN at 0.2450 ± 0.0407 . Even so, GraphResGNN still remains the best model. Therefore, ENZYMES is the

cleanest topic-facing dataset for arguing that cross-residual pathways can recover useful information that plain stacking loses, but not for arguing that cross should replace residual baselines outright.

6.5 Ablation Conclusions

Taken together, these dataset-level observations can now be condensed into a single ablation message.

The final ablation story is therefore precise:

- **Plain** → **Cross**: cross-residual routing is usually beneficial relative to plain stacking
- **Residual** → **Cross**: the gain is conditional and dataset-dependent
- **Node-level versus graph-level**: node-level cross is more stable, while graph-level cross achieves stronger wins only on selected datasets

This is the strongest claim that the current evidence supports. Cross-residual design should be presented as a meaningful and reusable architectural design space, not as a universal winner over residual reuse.

6.6 Parameter Sensitivity

The next question is whether this ablation story is stable under moderate changes in the cross-model hyperparameters.

To complement the structural ablation, we ran a focused parameter-sensitivity study for the two cross-residual models on the topic-facing datasets PROTEINS, DD, and ENZYMES. We varied depth, dropout, and learning rate under the same general training setting. Figures 6–8 summarize the resulting trends.

The sensitivity patterns are dataset-specific rather than uniform. On **PROTEINS**, NodeCrossGNN is most sensitive to depth, while GraphCrossGNN benefits from lighter regularization. On **DD**, both cross variants are more optimization-sensitive, and GraphCrossGNN reacts especially strongly to depth. On **ENZYMES**, however, the scan exposes a different regime: the validation signal is noticeably noisier, which makes local hyperparameter improvements less reliable.

To check whether the ENZYMES local improvements were stable, we also ran a follow-up evaluation using the most promising settings suggested by the scan. The follow-up remained below the original full-suite ENZYMES cross results, so we do not replace the main table. Instead, the result supports a more careful interpretation: on the smallest dataset, validation noise can dominate local hyperparameter gains, and apparent improvements do not necessarily transfer to the 5-fold mean.

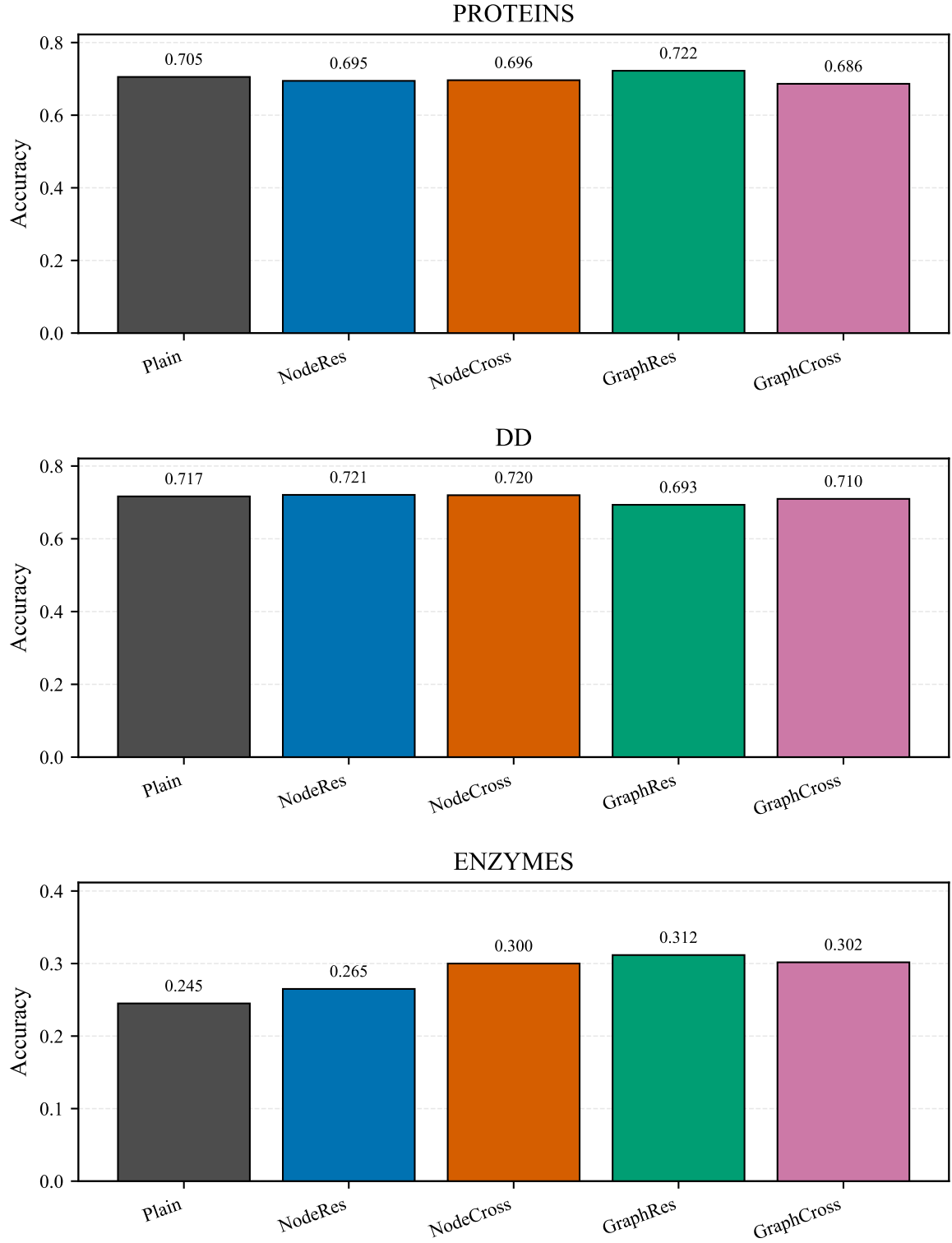


Figure 5: Focused comparison on the completed topic-facing TU datasets. The pattern is mixed rather than one-sided: graph-level residual refinement is strongest on PROTEINS and ENZYMES, while DD favors node-level reuse, with NodeCrossGNN remaining competitive but not dominant.

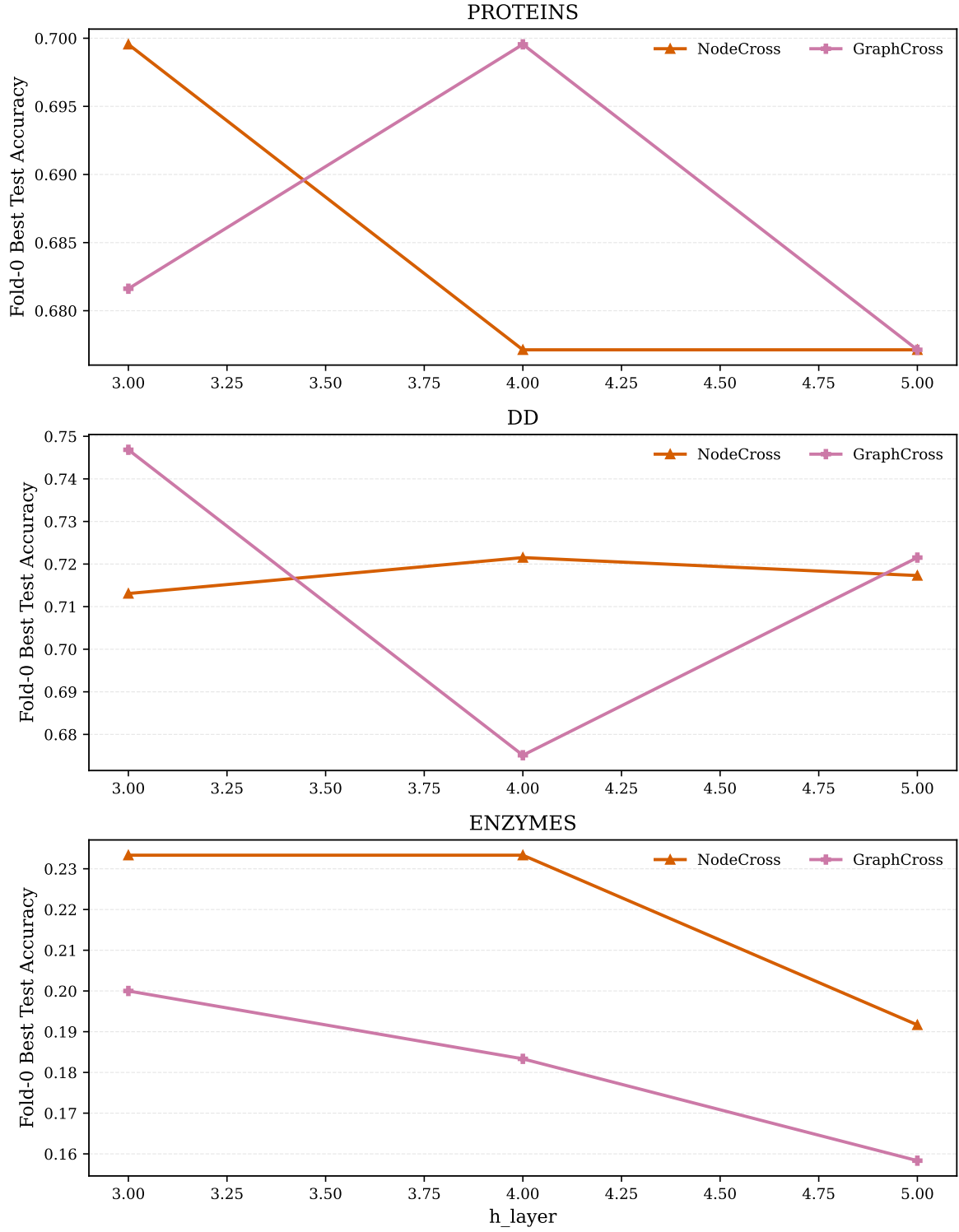


Figure 6: Depth sensitivity of the cross-residual models on the topic-facing datasets. NodeCrossGNN is most depth-sensitive on PROTEINS, while GraphCrossGNN on DD benefits strongly from the shallower three-layer setting.

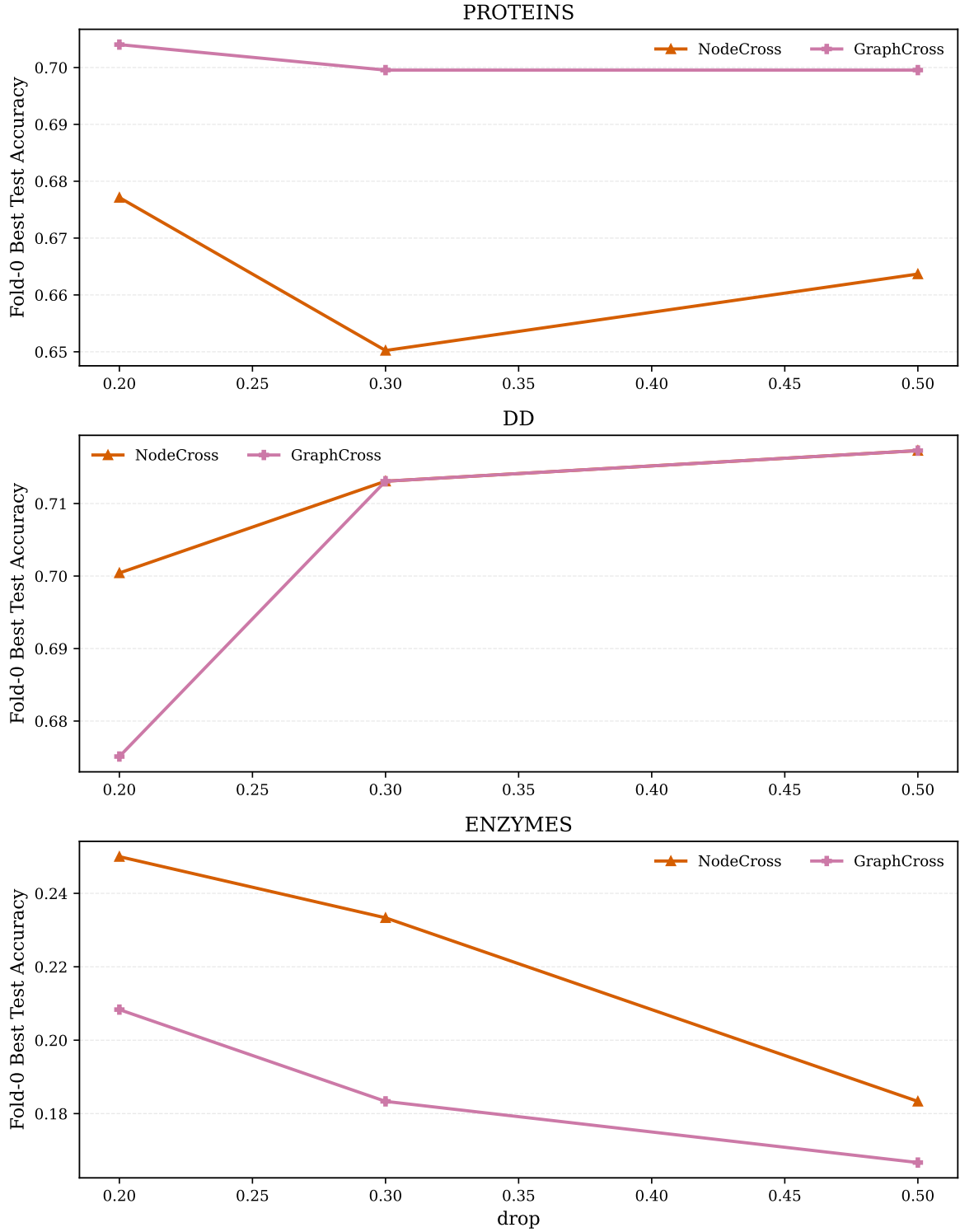


Figure 7: Dropout sensitivity of the cross-residual models. PROTEINS favors lighter regularization for GraphCrossGNN, DD prefers moderate-to-strong regularization, and ENZYMES remains unstable across all tested dropout settings.

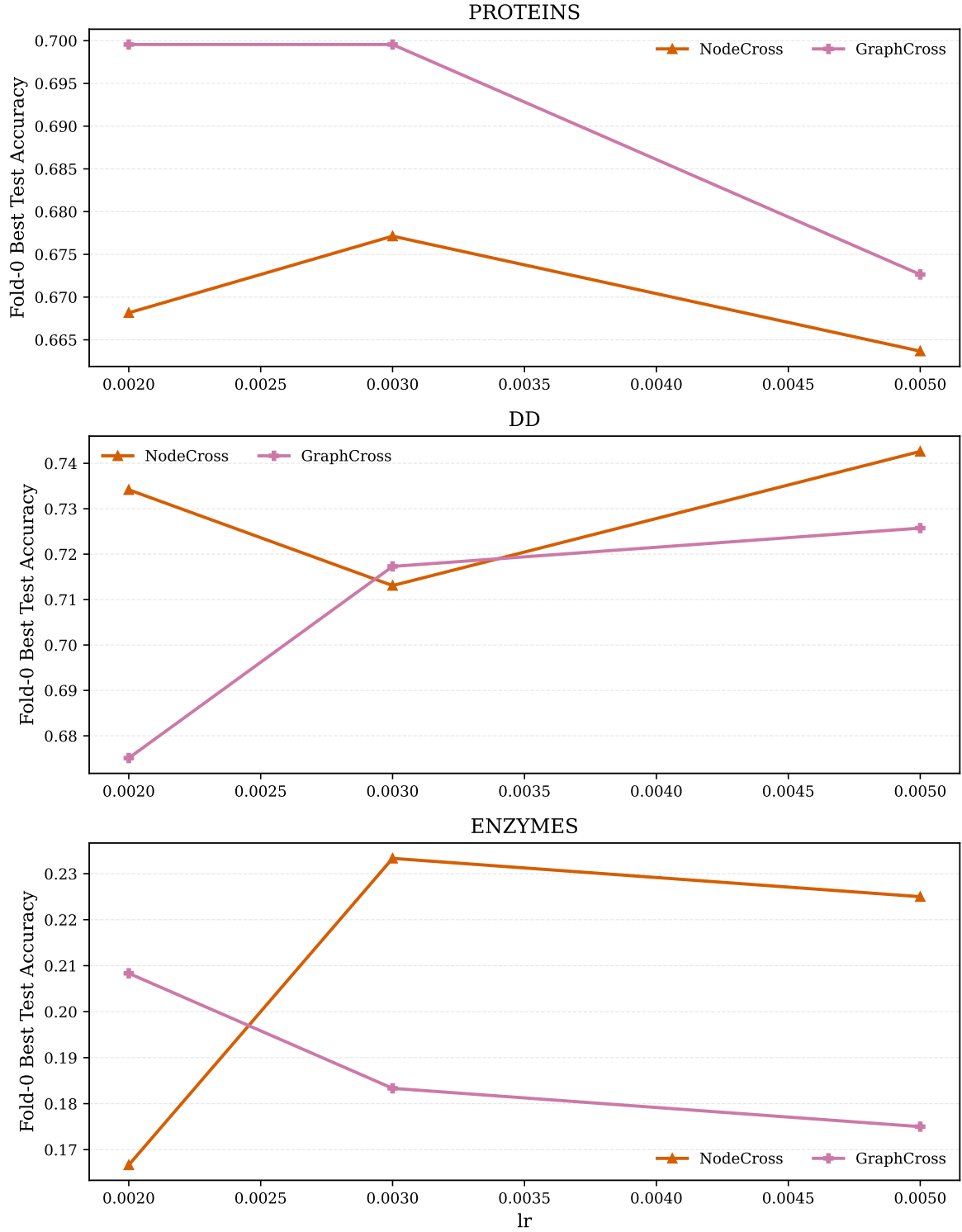


Figure 8: Learning-rate sensitivity of the cross-residual models. DD shows the clearest optimization dependence, with both cross variants improving under a more aggressive learning rate schedule.

6.7 Current Limitations

The experiments are rigorous within the chosen benchmark scope, but they do not remove the core scope limitations of the current study.

The experiments remain structure-centric rather than fully omics-integrative. Accordingly, the strongest defensible claim of the paper is that CR-GNN is an effective biomolecular graph representation architecture with dataset-dependent cross-residual gains. The current evidence does not yet justify a broader claim about integrative omics prediction or direct discovery of novel genes, domains, or motifs, but the revised benchmark package still has a clear extension path toward plant-related graph inference settings.

7 Conclusion

This paper reframed the cross-residual GNN study around biomolecular graph representation learning rather than generic benchmark breadth. The final result is a narrower but stronger claim: cross-residual interaction is useful, yet residual variants remain the strongest default family in the reported benchmark suite.

Accordingly, the main contribution is not a universally dominant model, but a controlled architecture family that makes these trade-offs visible. Residual models win on PROTEINS, DD, ENZYMES, and AIDS, while cross-residual models win on MUTAG and Mutagenicity; across the broader suite, the best cross variant still improves over the plain baseline on most datasets. The strongest defensible conclusion is therefore a dataset-dependent advantage for cross-residual routing, not a universal superiority claim over residual reuse.

For the topic-facing biological interpretation, the most relevant datasets remain PROTEINS, DD, and ENZYMES. On these benchmarks, GraphResGNN is strongest on PROTEINS and ENZYMES, whereas NodeResGNN is strongest on DD. Cross variants still provide useful gains over plain stacking, especially on ENZYMES, but they do not replace residual baselines as the default point of reference. This is the right level of claim for the current evidence and provides a credible methodological basis for future plant-oriented graph inference, where preserving both local functional motifs and graph-level biological context is equally important.

There are three immediate directions for future work. First, the topic fit can be strengthened by extending the same cross-residual design to richer protein and gene prediction settings. Second, the present graph-classification backbone can be extended toward plant-related graph inference tasks, especially plant protein identification and functional annotation settings, so that the architectural motivation outlined in the introduction is tested under explicit plant biological supervision. Third, the architecture-level findings reported here can be paired with stronger biological features, including sequence-derived embeddings, ontology signals, expression measurements, or other omics-derived inputs, to move beyond structure-only benchmarks.

A Why Residual Reuse Can Slow Over-smoothing

This appendix provides a simplified derivation that explains why residual reuse can mitigate over-smoothing. The goal is not to claim a full theorem for the nonlinear models in Section 4,

but to show that the proposed residual pathways change the effective propagation filter from a single high-order power to a lower-order polynomial mixture.

A.1 Linearized Plain Propagation

Consider the standard linearized message-passing update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}\mathbf{H}^{(\ell)}, \quad (21)$$

where $\mathbf{S}$ is a normalized graph propagation operator and $\mathbf{H}^{(0)} = \mathbf{X}$. Ignoring nonlinearities and trainable weights yields

$$\mathbf{H}^{(\ell)} = \mathbf{S}^\ell \mathbf{X}. \quad (22)$$

If $\mathbf{S}$ is diagonalizable as $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}$, then

$$\mathbf{H}^{(\ell)} = \mathbf{U}\mathbf{\Lambda}^\ell \mathbf{U}^{-1} \mathbf{X}. \quad (23)$$

For normalized propagation, the dominant eigenvalue is typically close to 1, while the remaining eigenvalues satisfy $|\lambda_i| < 1$. As ℓ grows, the factors λ_i^ℓ suppress non-dominant components, so node embeddings gradually collapse toward a low-dimensional smooth subspace. This is the standard over-smoothing effect: nodes become increasingly difficult to distinguish because repeated propagation attenuates the higher-frequency components that encode local differences.

A.2 Node-Level Residual Reuse

Now consider a residual reuse update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}(\mathbf{H}^{(\ell)} + \beta \mathbf{H}^{(\ell-1)}), \quad (24)$$

where $\beta \geq 0$ controls the strength of the reused hidden state. In this study, we set $\beta = 1$.

Define $\mathbf{H}^{(\ell)} = P_\ell(\mathbf{S})\mathbf{X}$ with $P_0(\mathbf{S}) = \mathbf{I}$ and $P_1(\mathbf{S}) = \mathbf{S}$. Then Eq. (24) implies the polynomial recurrence

$$P_{\ell+1}(\mathbf{S}) = \mathbf{S}(P_\ell(\mathbf{S}) + \beta P_{\ell-1}(\mathbf{S})). \quad (25)$$

The first few terms are

$$P_1(\mathbf{S}) = \mathbf{S}, \quad (26)$$

$$P_2(\mathbf{S}) = \mathbf{S}^2 + \beta \mathbf{S}, \quad (27)$$

$$P_3(\mathbf{S}) = \mathbf{S}^3 + 2\beta \mathbf{S}^2, \quad (28)$$

$$P_4(\mathbf{S}) = \mathbf{S}^4 + 3\beta \mathbf{S}^3 + \beta^2 \mathbf{S}^2. \quad (29)$$

Therefore, residual reuse no longer applies only the single power $\mathbf{S}^\ell$; instead, it produces a mixture of lower-order and higher-order propagation terms. In spectral coordinates, each eigen-component evolves as

$$z_i^{(\ell+1)} = \lambda_i(z_i^{(\ell)} + \beta z_i^{(\ell-1)}), \quad (30)$$

which is fundamentally different from the plain recurrence $z_i^{(\ell)} = \lambda_i^\ell z_i^{(0)}$. The plain model attenuates every non-dominant mode with a pure monomial factor, whereas the residual model preserves a polynomial combination of powers of λ_i . As a result, informative mid-frequency components are not suppressed as aggressively at the same depth.

A.3 Implication for Over-smoothing

The derivation above suggests the following interpretation. Plain stacking behaves like a repeated low-pass filter whose order increases monotonically with depth. Residual reuse changes that filter into a polynomial response that still smooths the graph, but preserves more low-order propagation information from earlier layers. This does not eliminate smoothing, yet it slows the rate at which node representations collapse to an almost indistinguishable subspace. In other words, residual reuse can improve depth tolerance because it reduces the dominance of the highest propagation order.

A.4 Graph-Level Residual Passing

The same intuition extends to graph-level residual propagation. In the graph-conditioned variants, a pooled summary from the previous block is broadcast back to the current block, so the hidden state is no longer determined only by repeated applications of $\mathbf{S}$. A simplified affine form is

$$\mathbf{H}_t^{(\ell+1)} = \mathbf{S}\mathbf{H}_t^{(\ell)} + \mathbf{1}(\mathbf{g}_{t-1})^\top, \quad (31)$$

where $\mathbf{g}_{t-1}$ is the preceding graph summary. The additive term injects a lower-order global context at every block and prevents the representation from being governed solely by deeper powers of the propagation operator. This offers a second pathway for delaying over-smoothing, although the empirical sections show that graph-level reuse is more sensitive to dataset structure and computational cost than the lighter node-level residual design.

A.5 Scope of the Derivation

This appendix uses a linearized analysis and therefore abstracts away nonlinear activations, dropout, optimizer dynamics, and operator-specific details. It should be read as an explanatory mechanism rather than as a complete proof. Still, it aligns with the empirical observation that node-level residual reuse remains more stable than plain stacking as depth increases, and it provides a compact theoretical rationale for why reusing earlier states can alleviate over-smoothing.

B Complete Benchmark Tables

For completeness, we reproduce the final full-suite benchmark tables in the appendix. These are the same results discussed in Section 6, but collecting them here makes the full comparison easier to inspect during review.

Table 7: Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

Table 8: Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486	0.8455 $\pm$ 0.0257	0.7814 $\pm$ 0.0235
NodeResGNN	0.7343 $\pm$ 0.0734	0.8690 $\pm$ 0.0354	0.7897 $\pm$ 0.0139
NodeCrossGNN	0.7397 $\pm$ 0.0676	0.8735 $\pm$ 0.0412	0.8033 $\pm$ 0.0159
GraphResGNN	0.7290 $\pm$ 0.0585	0.9160 $\pm$ 0.0121	0.8022 $\pm$ 0.0220
GraphCrossGNN	0.7129 $\pm$ 0.0539	0.8710 $\pm$ 0.0412	0.8029 $\pm$ 0.0182

Table 9: Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

References

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [2] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *International Conference on Machine Learning*, 2017, pp. 1263–1272.
- [3] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in neural information processing systems*, 2017, pp. 1024–1034.
- [4] V. Gligorijevic, P. D. Renfrew, T. Kosciolk, J. K. Leman, D. Berenberg, T. Vatanen, C. Chandler, B. C. Taylor, I. M. Fisk, H. Vlamakis, R. J. Xavier, R. Knight, K. Cho, and R. Bonneau, “Structure-based protein function prediction using graph convolutional networks,” *Nature Communications*, vol. 12, no. 1, 2021. [Online]. Available: <http://dx.doi.org/10.1038/s41467-021-23303-9>
- [5] R. You, S. Yao, H. Mamitsuka, and S. Zhu, “Deepgraphgo: graph neural network for large-scale, multispecies protein function prediction,” *Bioinformatics*, vol. 37, no. Suppl. 1, pp. i262–i271, 2021. [Online]. Available: <http://dx.doi.org/10.1093/bioinformatics/btab270>
- [6] M. Reau, N. Renaud, L. C. Xue, and A. M. J. J. Bonvin, “Deeprank-gnn: a graph neural network framework to learn patterns in protein–protein interfaces,” *Bioinformatics*, vol. 39, no. 1, 2022. [Online]. Available: <http://dx.doi.org/10.1093/bioinformatics/btac759>
- [7] J. Sui, J. Chen, Y. Chen, N. Iwamori, and J. Sun, “Identification of plant vacuole proteins by using graph neural network and contact maps,” *BMC Bioinformatics*, vol. 24, no. 1, pp. 1–16, 2023.
- [8] K. Charulekha, A. Girigoswami, K. Girigoswami, M. Manoj, P. Pallavi, P. Gowtham, A. A. Robert, and A. V. Anand, “Uncovering complicated plant biological networks through the assistance of artificial intelligence tools,” *AI in Plant Science and Precision Agriculture*, pp. 25–54, 2025.
- [9] T. G. B. Ngo, C. Liseron-Monfils, S. Das, J. Ubbens, P. Ashe, and D. Konkin, “Improving plant functional annotation from knowledge graphs using graph neural networks,” *bioRxiv*, 2025.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [11] X. Bresson and T. Laurent, “Residual gated graph convnets,” *arXiv preprint arXiv:1711.07553*, 2017.
- [12] L. Zhao and L. Akoglu, “Pairnorm: Tackling over-smoothing in gns,” *arXiv preprint arXiv:2009.10698*, 2020.

- [13] T. Cai, J. Luo, S. Wang, K. Zhang, Y. Tian, L. Yang, Z. Li, and K. Weinberger, “Graphnorm: A principled approach to accelerating graph neural network training,” in *International Conference on Machine Learning*, 2021, pp. 1277–1287.
- [14] Y. Rong, W. Huang, T. Xu, and J. Huang, “Dropedge: Towards deep graph convolutional networks on node classification,” *arXiv preprint arXiv:1907.10903*, 2019.
- [15] K. Xu, C. Li, Y. Tian, X. Du, J. Leskovec, and S. Jegelka, “Representation learning on graphs with jumping knowledge networks,” in *International Conference on Machine Learning*, 2018, pp. 5449–5458.
- [16] C. Morris, R. Wattenhofer, and K. Kersting, “Tudataset: A collection of benchmark datasets for learning with graphs,” *arXiv preprint arXiv:2007.08663*, 2020.
- [17] M. Gori, G. Monfardini, and F. Scarselli, “A new model for learning in graph domains,” *Proceedings of IJCNN*, 2005.
- [18] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, “Convolutional networks on graphs for learning molecular fingerprints,” in *Advances in neural information processing systems*, 2015, pp. 2224–2232.
- [19] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, “Hierarchical graph representation learning with differentiable pooling,” in *Advances in neural information processing systems*, 2018, pp. 4800–4810.
- [20] J. Lee, I. Lee, and J. Kang, “Self-attention graph pooling,” *arXiv preprint arXiv:1904.08082*, 2019.
- [21] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Liò, “Towards sparse hierarchical graph classifiers,” in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2018, pp. 165–176.
- [22] C. Liu, X. Wang, Y. Li, Y. Wang, and J. Gao, “Graph pooling for graph neural networks,” in *International Joint Conference on Artificial Intelligence*, 2023, pp. 4254–4261.
- [23] Z. Li, H. Chen, Z. Zeng, J. He, X. Mao, N. Lv, and L. Chen, “Graph pooling for graph-level representation learning: a survey,” *Artificial Intelligence Review*, vol. 57, no. 8, pp. 1–51, 2024.
- [24] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *International Conference on Learning Representations (ICLR)*, 2019.
- [25] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [26] M. Zhang, Y. Liu, Q. Chen, and S. Pan, “Personalized graph neural networks with attention mechanism,” *arXiv preprint arXiv:1910.08887*, 2019.
- [27] V. P. Dwivedi and X. Bresson, “A generalization of transformer networks to graphs,” *arXiv preprint arXiv:2012.09699*, 2020.

- [28] D. Beaini, S. Passaro, E. Létouzey, J. Domke, S. Erdos, P. Lió, P. Veličković, and M. M. Bronstein, “Directional graph networks,” *arXiv preprint arXiv:2010.02863*, 2020.
- [29] E. Rossi, F. Frasca, F. M. Bianchi, N. Alletto, and P. Lio, “Edge directionality improves learning on heterophilic graphs,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [30] F. M. Bianchi, F. Grattarola, and C. Alippi, “Graph neural networks with convolutional arma filters,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- [31] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, G. Steeg, and A. Galstyan, “Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing,” in *International Conference on Machine Learning*, 2019, pp. 33–42.
- [32] A. Gravina, N. Alletto, A. Rossi, F. M. Bianchi, and P. Lio, “Anti-symmetric dgn: A stable architecture for deep graph networks,” *arXiv preprint arXiv:2210.09789*, 2022.
- [33] G. Li, E. Muller, A. Thabet, and B. Ghanem, “Deeper insights into graph convolutional networks for semi-supervised learning,” *arXiv preprint arXiv:1809.02584*, 2018.
- [34] Y. Chen, X. Wei, P. Xu, X. Wang, P. Luo, Z. Li, and J. Tang, “Revisiting over-smoothing in deep gcns,” *arXiv preprint arXiv:2003.13663*, 2020.
- [35] S. Li, M. Zhang, M. Jin, H. Zhang, R. Ying, Z. Li, and J. Yan, “A non-asymptotic analysis of oversmoothing in graph neural networks,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [36] J. Topping, F. Di Giovanni, B. Chamberlain, M. Bronstein, D. Vendrig, and P. Lio, “Understanding over-squashing and bottlenecks on graphs via curvature,” *arXiv preprint arXiv:2111.14522*, 2021.
- [37] U. Alon and E. Yahav, “On the bottleneck of graph neural networks,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [38] Y. Li, X. Wang, Z. Wang, and J. Li, “Learning graph normalization for graph neural networks,” *Neurocomputing*, vol. 486, pp. 140–151, 2022.
- [39] G. Li, E. Müller, A. Thabet, and B. Ghanem, “Training graph neural networks with 1000 layers,” in *International Conference on Machine Learning*, 2021, pp. 6403–6413.
- [40] K. Oono and T. Suzuki, “Simple and deep graph convolutional networks,” *arXiv preprint arXiv:2006.13604*, 2020.
- [41] G. Wang, I. Balazevic, M. Znis, M. Gitta, Y. Kwon, M. Seeger, X. Li, L. De Raedt, and M. Welling, “Multi-hop attention graph neural networks,” in *International Joint Conference on Artificial Intelligence*, 2021, pp. 2112–2120.
- [42] X. Sun, R. Wang, L. Cui, X. Kong, and S. Li, “Attention-based graph neural networks: a survey,” *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13 493–13 539, 2023.

- [43] J. Cai, X. Zheng, C. Luo, C. Yang, and E. Santus, “Understanding graph embedding methods and their applications,” *arXiv preprint arXiv:2012.08019*, 2020.
- [44] H. Du, Y. Zhang, P. Zhang, B. Gu, X. Wei, J. Guo, A. Li, Z. Wang, T. Ma *et al.*, “Densegnn: universal and scalable deeper graph neural networks for high-performance property prediction in crystals and molecules,” *npj Computational Materials*, vol. 10, no. 1, p. 304, 2024.
- [45] H. Van Pham, U. The, T. Do, and A. Nguyen, “Hierarchical pooling in graph neural networks to enhance graph classification,” *Applied Sciences*, vol. 11, no. 17, p. 8164, 2021.
- [46] H. Gao and S. Ji, “Graph u-net,” *arXiv preprint arXiv:1905.05178*, 2019.
- [47] J. Klicpera, A. Bojchevski, and S. Günnemann, “Combining label propagation and simple models out-performs graph neural networks,” *arXiv preprint arXiv:1810.05997*, 2018.
- [48] L. Zhao, Z. Leng, Y. Chen, and L. Akoglu, “Tackling over-smoothing for general graph learning,” *arXiv preprint arXiv:2008.09864*, 2020.
- [49] M. Fey and J. E. Lenssen, “Fast graph representation learning with pytorch geometric,” *arXiv preprint arXiv:1903.02428*, 2019.